Fundamentals of Artificial Intelligence, Machine and Deep Learning Study Sheet

Amina Hanid - Sarra Lajnef
Based on: Course FAI-polito

April 21, 2026

Contents

1 Lecture 1: Introduction to AI, ML and DL **2**

2 Lecture 2: Probability and Statistics Bootcamp **4**
2.1 The Product Chain Rule . . . 4
2.2 Independence . . . 4
2.3 Bayes' Rule . . . 4
2.4 The Naïve Bayes Classifier: From Decision to Probability . . . 4
2.5 Parameter Estimation . . . 6
2.6 Estimator Properties . . . 6
2.7 Bias-Variance Decomposition: . . . 7
2.8 Fundamental Laws . . . 8
2.8.1 Law of Large Numbers (LLN) . . . 8
2.8.2 Central Limit Theorem (CLT) . . . 8
2.8.3 Applications in Machine Learning . . . 9

3 Lecture 3: How to Build a Deep Learning Pipeline **10**
3.1 Defining Objectives & Computer Vision Tasks . . . 10
3.2 Data Collection and Preprocessing . . . 10
3.3 Model Design . . . 10
3.4 Training and Evaluation . . . 10
3.5 Improving through Experimentation . . . 11
3.6 Final Step: Save Your Model . . . 11

4 Lecture 3bis: Intro to Learning Theory **11**

1 Lecture 1: Introduction to AI, ML and DL

1. What is AI? Approaches to Artificial Intelligence

AI is broadly defined as making computers think like a human. The field looks at this through different lenses:

Thinking Humanly:

Acting Humanly (The Turing Test Approach):

- **Natural language processing:** to communicate and generate comprehensible sentences.
- **Knowledge representation:** to store knowledge.
- **Automated reasoning:** to answer questions and draw conclusions.
- **Machine learning:** to detect/extract patterns, adapt to new circumstances, and improve effective behavior.
- **Computer vision:** to perceive objects.
- **Robotics:** to manipulate objects and interact beyond language.

Issues with the Turing Test:

- Success depends on deception.
- Protocols and judges vary.
- Chatbots can perform well using "cheap tricks".

The Winograd Schema: Designed to overcome Turing Test weaknesses. These are multiple-choice questions that people can answer easily, but computers cannot answer using simple "cheap tricks".

Thinking Rationally (The Logicist Approach):

Acting Rationally (AI = Computational Rationality):

2. Machine Learning Foundations

Definition of Learning: An agent learns when its performance (P) at a specific task (T) improves with experience/data (E). We want the computer to do well without manually building its competence.

Classical Programming vs. Machine Learning: **Classical Programming:** You input Data + Program (Rules) into the computer, and it produces the Output.

Machine Learning (Programming with Data): Rule-based implementation is brittle, difficult to maintain, and misses edge cases. Instead, we input Data + Output (Answers), and the learning algorithm produces the Program (Rules/Model).

ML Pseudo-Algorithm:

1. Understand domain, prior knowledge, and goals.
2. Data integration, selection, cleaning, and pre-processing.
3. Learn models.
4. Interpret results.
5. Consolidate and deploy discovered knowledge.

3. The AI / ML / DL Hierarchy

AI vs. ML vs. DL:

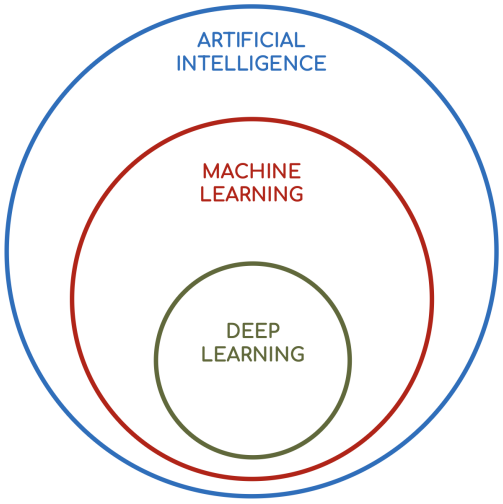

Figure 1: AI vs. ML vs. DL

Main ML Paradigms:

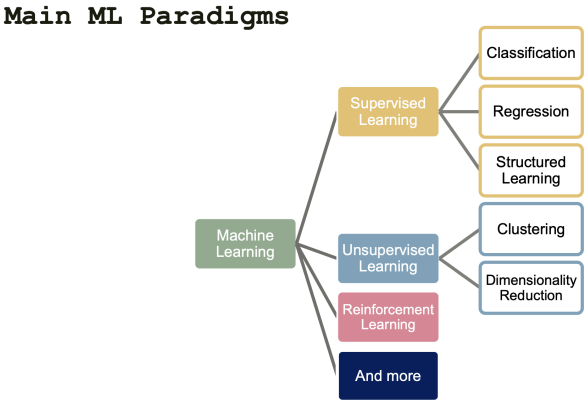

Figure 2: Main ML Paradigms

2 Lecture 2: Probability and Statistics Bootcamp

2.1 The Product Chain Rule

Product Rule:

$$P(y)P(x|y) = P(x, y)$$

Chain Rule (General Form):

$$P(x_1, x_2, \ldots, x_n) = \prod_{i=1}^{n} P(x_i|x_1, \ldots, x_{i-1})$$

Chain Rule (Three Variables):

$$P(x_1, x_2, x_3) = P(x_1)P(x_2|x_1)P(x_3|x_1, x_2)$$

2.2 Independence

Independent Random Variables (Joint):

$$P(X, Y) = P(X)P(Y)$$

Independent Random Variables (Conditional):

$$P(X|Y) = P(X)$$

2.3 Bayes' Rule

Bayes' Rule:

$$P(x|y) = \frac{P(y|x)}{P(y)}P(x)$$

Bayesian Proportionality (MAP:Maximum a Posteriori): P(y) = constant (because given) tikz amsmath

likelihood

$$P(x|y) \propto \frac{P(y|x)}{P(y)}\cancel{P(x)}$$

Ignored in f_{NB}

posterior **prior**

2.4 The Naïve Bayes Classifier: From Decision to Probability

The Naïve Bayes classifier is a generative learning model based on Bayes' Theorem, utilizing a strong simplifying assumption of independence.

1. The "Naïve" Assumption

It is assumed that each attribute X_i is conditionally independent of all other attributes given the class label Y. This simplifies the joint likelihood into a product of individual likelihoods:

$$P(X_1, \ldots, X_d|Y) = \prod_{i=1}^{d} P(X_i|Y)$$

2. The Decision Rule: $f_{NB}(x)$

The primary goal of the classifier is to predict the class label. To achieve this, we seek the class y that maximizes the product of the prior and the likelihood.

The decision function $f_{NB}(x)$ is defined as:

$$f_{NB}(x) = \arg\max_y \left[P(y) \prod_{i=1}^{d} P(x_i|y) \right]$$

Note: The denominator $P(x)$ is ignored here because it is identical for all classes and does not change the choice of the "winner".

3. Calculating the Posterior Probability: $P(y|x)$

If we wish to know the level of certainty (the actual probability between 0 and 1), we must normalize the scores[cite: 355, 370]. The complete formula is:

$$P(Y = y_k|X) = \frac{P(y_k) \prod_{i=1}^{d} P(x_i|y_k)}{P(x)}$$

4. The Normalization Factor α

In practice, $P(x)$ is often difficult to calculate directly. We use a normalization factor α to ensure the sum of probabilities across all classes equals 1:

1. **Calculate the unnormalized score** for each class k:

$$\tilde{P}_k = P(y_k) \prod_{i=1}^{d} P(x_i|y_k)$$

2. **Calculate** α (the inverse of the sum of all scores):

$$\alpha = \frac{1}{\sum_k \tilde{P}_k}$$

3. **Final Probability**:

$$P(y_k|x) = \alpha \tilde{P}_k$$

Summary of Steps

- **Step 2 (Decision)**: We compare the scores $\tilde{P}_k$. The largest determines $f_{NB}(x)$. This is the **action**.
- **Step 4 (Normalization)**: We use α to transform these scores into probabilities $P(y|x)$. This is the **measure of confidence**.

5. Naïve Bayes Classifier Summary:

Naïve Bayes Decision Rule:

Given:

- Class prior P(y)
- d conditionaly independant features $X_1, ..., X_d$ given the class label Y
- For each X_i feature, we have the conditional likelihood $P(X_i|Y)$

$$f_{NB}(x) = \arg\max_y P(y) \prod_{i=1}^{d} P(x_i|y)$$

- **Computationally very fast**
  - Training: only one pass over the training set
  - Classification: linear in the number of attributes
- Despite its conditional independence assumption, Naïve Bayes classifier shows a good performance in several application domains.

When to use?

- A moderate or large training set available with instances represented by a large number of attributes.

Issue: It might happen that an attribute never appears for a specific class, e.g., `Temperature=cold` never appears together with `EnjoySpt=yes` $\longrightarrow P(\text{cold}|\text{yes}) = 0 \longrightarrow P(\text{yes}|\text{cold}) = 0$.

Laplacian Correction: A trick used if a specific attribute never appears with a specific class in the training data (which would yield a 0 probability). It adds "virtual" samples to prevent the entire probability product from becoming zero. Add one or more "virtual" samples with (`cold`, `yes`) and one with (`warm`, `yes`).

$$\hat{P}(X_j = a_{jk} \mid C = c_i) = \frac{n_c + mp}{n + m}$$

n_c : number of training examples for which $X_j = a_{jk}$ and $C = c_i$

n : number of training examples for which $C = c_i$

p : prior estimate (usually, $p = 1/t$ for t possible values of X_j)

m : weight to prior (number of "virtual" examples, $m \geq 1$)

Figure 3: laplacien

2.5 Parameter Estimation

Maximum Likelihood Estimation (MLE):(page 36 lecture 2 Intro proba)
"Given a model, which parameter best explains the data I saw?" + No prior knowledge is considered

$$\hat{\theta}_{MLE} = \arg\max_{\theta} P(D|\theta)$$

Maximum a Posteriori (MAP) Estimation:
It also considers prior beliefs about the parameter

$$\hat{\theta}_{MAP} = \arg\max_{\theta} P(D|\theta)P(\theta)$$

MLE for Gaussian Mean:

$$\hat{\mu}_{MLE} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

MLE for Gaussian Variance:

$$\hat{\sigma}^2_{MLE} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{\mu})^2$$

2.6 Estimator Properties

Bias: how close is the estimate to the true value ?
Variance: how much does the estimate change for different runs (eg. different datasets)?
Bias of an Estimator:

$$bias(\hat{\theta}_m) = E[\hat{\theta}_m] - \theta$$

The estimator is **unbiased** if $bias(\hat{\theta}_m) = 0$
Estimator of Gaussian Variance: (page 47 lecture 2 intro proba)

$$\hat{\sigma}^2_m = \frac{1}{m-1}\sum_{i=1}^{m}(x^{(i)} - \hat{\mu}_m)^2$$

The Variance of an Estimator

1. Definition and Intuition

The variance indicates how much we expect the estimator to vary as a function of different data samples from the same distribution.

2. Efficiency and Comparison

If two estimators of a parameter are both unbiased, the better one is the one with the least amount of variability.

- An estimator $\hat{\theta}_1$ is said to be **more efficient** than $\hat{\theta}_2$ if $Var(\hat{\theta}_1) < Var(\hat{\theta}_2)$.

3. Standard Error (SE)

The square root of the variance is called the **standard error**, denoted as $SE(\hat{\theta})$.

- It measures how we expect the estimate to vary when obtaining different samples from the same distribution.
- **Standard Error of the Mean:** For a dataset of size m, it is given by:

$$SE(\hat{\mu}_m) = \sqrt{Var\left[\frac{1}{m}\sum_{i=1}^{m} x^{(i)}\right]} = \frac{\sigma}{\sqrt{m}}$$

- *Note:* As the number of samples m increases, the accuracy of the prediction improves as the SE decreases.

4. Confidence Intervals and Decision Making

Since the mean is normally distributed (due to the Central Limit Theorem), we can compute the probability that the true expectation falls in a specific interval.

- **95% Confidence Interval:** Centered on the mean $\hat{\mu}_m$ is given by:

$$(\hat{\mu}_m - 1.96SE(\hat{\mu}_m), \hat{\mu}_m + 1.96SE(\hat{\mu}_m))$$

- **Comparison Rule:** Algorithm A is better than B if the error upper bound of A is less than the lower bound of B.

Mean Squared Error (MSE):

$$MSE(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \theta)^2]$$

2.7 Bias-Variance Decomposition:

Let $\hat{\theta}$ be an estimator for an unknown parameter θ. The **Mean Squared Error (MSE)** of this estimator can be decomposed into the square of the bias and the variance:

Formula:

$$MSE(\hat{\theta}) = b^2(\hat{\theta}) + Var(\hat{\theta})$$

Proof:

$$MSE(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \theta)^2]$$

$$MSE(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}] + \mathbb{E}[\hat{\theta}] - \theta)^2]$$

$$MSE(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2 + (\mathbb{E}[\hat{\theta}] - \theta)^2 + 2(\hat{\theta} - \mathbb{E}[\hat{\theta}])(\mathbb{E}[\hat{\theta}] - \theta)]$$

$$MSE(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2] + (\mathbb{E}[\hat{\theta}] - \theta)^2 + 2\mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])](\mathbb{E}[\hat{\theta}] - \theta)$$

- The first term is the definition of Variance: $\mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2] = Var(\hat{\theta})$.

- The second term is the square of the Bias: $(\mathbb{E}[\hat{\theta}] - \theta)^2 = b^2(\hat{\theta})$.
- The third term vanishes because $\mathbb{E}[\hat{\theta} - \mathbb{E}[\hat{\theta}]] = \mathbb{E}[\hat{\theta}] - \mathbb{E}[\hat{\theta}] = 0$.

Result:

$$MSE(\hat{\theta}) = Var(\hat{\theta}) + b^2(\hat{\theta})$$

Comparison: Generative vs. Discriminative Models

Feature	**Generative Models (MAP)**	**Discriminative Models (MLE)**
Goal	Estimate the joint probability $P(X, Y)$.	Estimate the conditional probability $P(Y\|X)$ directly.
Approach	Model how the data was generated: $P(X\|Y)P(Y)$.	Model the decision boundary between classes.
Bayes' Rule	Essential to infer $P(Y\|X)$ from the learned joint distribution.	Not required, as $P(Y\|X)$ is learned directly.
Prior Knowledge	Easy to incorporate via prior distributions $P(Y)$ or $P(\theta)$.	Harder to incorporate prior information about the data distribution.
Data Complexity	Struggles with very complex distributions (e.g., high-res images).	Superior for complex data like text, images, or movies.
Small Samples	Generally more robust (Bayesian/MAP approach is "better when samples are small").	May perform poorly or overfit with very few samples.
Bias / Variance	**High Bias** (due to strong assumptions like Naïve Bayes) / **Low Variance**.	**Low Bias** / **Higher Variance** (requires more data to stabilize the boundary).
Example	Naïve Bayes.	Logistic Regression, SVM, Neural Networks.

2.8 Fundamental Laws

2.8.1 Law of Large Numbers (LLN)

The Law of Large Numbers states that as the number of identically distributed, independent variables (i.i.d.) increases, their sample mean converges to the true theoretical mean of the population.

Definition

Let $X_1, X_2, \ldots, X_n$ be a sequence of i.i.d. random variables with a finite expected value $E[X_i] = \mu$. The sample mean is defined as:

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^{n} X_i$$

As $n \to \infty$, the sample mean $\bar{X}_n$ converges to the population mean μ:

$$\bar{X}_n \xrightarrow{p} \mu$$

2.8.2 Central Limit Theorem (CLT)

The Central Limit Theorem describes the *distribution* of the sample mean. It states that, regardless of the original distribution of the variables (as long as it has finite variance), the distribution of the sample mean will approach a Normal (Gaussian) distribution as the sample size becomes large.

Theorem Details

Given a set of n i.i.d. random variables with mean μ and variance σ^2, the distribution of their sum (or mean) tends toward:

$$Z_n = \frac{1}{\sqrt{n}} \sum_{i=1}^{n} \frac{X_i - \mu}{\sigma} \approx \mathcal{N}(0, 1)$$

Standard Error

The standard deviation of this sample mean distribution is known as the **Standard Error (SE)**:

$$SE(\bar{X}_n) = \frac{\sigma}{\sqrt{n}}$$

This formula shows that the estimate becomes more precise (less variable) as the dataset size n increases.

2.8.3 Applications in Machine Learning

- **Error Bounds:** Since the mean is normally distributed per the CLT, we can calculate the probability that a true expectation falls within a specific chosen interval.
- **Model Comparison:** An algorithm A is considered better than B if the error upper bound of A is strictly less than the lower bound of B.

3 Lecture 3: How to Build a Deep Learning Pipeline

Introduction A deep learning pipeline is a structured workflow designed to ensure meaningful and reproducible results. It follows a logical progression:

Define Task → Data Prep → Model Selection → Training & Evaluation → Experimentation → Deployment

3.1 Defining Objectives & Computer Vision Tasks

Deep learning models process data through **numerical encoding** to learn representations, patterns, and features.

Common Computer Vision Tasks

- **Image Classification**: Assigning a single label to an entire image (e.g., "Cat").
- **Object Detection**: Identifying objects and determining their location via bounding box coordinates.
- **Semantic Segmentation**: Identifying pixels belonging to each class (pixel-wise mask).
- **Instance Segmentation**: Distinguishing between individual objects of the same class (unique masks for different instances).

3.2 Data Collection and Preprocessing

Understanding Data Nature

- **Supervision Type**: Supervised (labeled), Unsupervised (unlabeled), or Semi-supervised (mixed).
- **Data Imbalance**: Overrepresented categories cause bias. *Solution*: Resampling or class balancing.
- **Dataset Size**: Small datasets lead to **overfitting** (memorization). *Solution*: Data Augmentation (flipping, rotating) or synthetic data.

Preprocessing & Normalization

Normalization ensures data is on a standard scale for stable learning. Images must also be processed to have fixed dimensions through resizing and cropping.

The Two Key Data Splits

The ultimate goal is **generalization** to unseen data.

- **Training Set**: Used for model learning; should be large and diverse.
- **Test Set**: Used only for final evaluation; must never be seen during training.

3.3 Model Design

Selection depends on:

- **Task Type**: (e.g., Classification vs. Segmentation).
- **Data Size**: Pretrained models for small data; training from scratch for large data.
- **Resources**: Memory and processing power (GPU/TPU).
- **Inference Speed**: Lightweight models are required for real-time robotics.

3.4 Training and Evaluation

- **Training**: Iterative update of parameters using a **loss function** until the error is minimized.
- **Evaluation**: Testing generalization to ensure the model hasn't just memorized the training data.

Common Metrics

- **Accuracy**: Percentage of correct predictions.
- **Precision & Recall**: Handling false positives vs. false negatives.
- **IoU (Intersection over Union)**: Measures overlap between predicted and ground truth masks/boxes.

3.5 Improving through Experimentation

Hyperparameters

Settings chosen before training begins:

- **Learning Rate**: Size of the parameter updates at each step.
- **Epochs**: Number of complete passes through the dataset.
- **Batch Size**: Samples processed before each parameter update.

Validation Strategies

- **Validation Set**: A separate set used to tune hyperparameters to avoid overfitting the test set.
- **K-Fold Cross-Validation**:
  1. Split data into K equal folds.
  2. Train K times, using a different fold as validation each time.
  3. Average the performance for a reliable estimate.

3.6 Final Step: Save Your Model

Ensure the trained parameters and architecture are stored for deployment.

4 Lecture 3bis: Intro to Learning Theory

Supervised Learning Foundations

- **Input/Output Spaces**: $\mathcal{X}$ (features) and $\mathcal{Y}$ (labels).
- **Joint Distribution D**: $P(X, Y)$ represents the unknown true process.
- **Hypothesis**: $h \in \mathcal{H}$ which is the classifier, where $h : \mathcal{X} \to \mathcal{Y}$.
- We define **the loss of the classifier** that mesures the error between the assigned prediction and the prediction of the correct lableing function $f : \mathcal{X} \to \mathcal{Y}$ where $f(\mathcal{X}) = \mathcal{Y}$
  L(h(X), f(X)) = L(h(X), Y) = $\begin{cases} 1, & \text{if } h(X) \neq Y \\ 0, & \text{if } h(X) = Y \end{cases}$ "0-1 Loss"

Risk Definitions

Expected Risk (True Risk): The True Risk of h is the probability that it does not predict the correct label on a random data point drawn from $\mathcal{D}$.

$\mathrm{R}_{\mathcal{D}}(h) := \mathbb{P}[h(X) \neq f(X)] = \mathbb{P}[h(X) \neq Y]$
$R_{\mathcal{D}}(h) = \mathbb{E}_{(x,y)\sim P}[\ell(h(x), y)]$

Empirical Risk (Training Risk):

$$\hat{L}_n(h) = \frac{1}{n} \sum_{i=1}^{n} \ell(h(x_i), y_i)$$

Empirical Risk Minimization (ERM): The learner chooses $\hat{h} = \arg\min_{h \in \mathcal{H}} \hat{L}_n(h)$.

Generalization and Complexity

VC Dimension (d_{VC}): Measures the capacity of $\mathcal{H}$. For large n, with probability $1-\delta$:

$$L(h) \leq \hat{L}_n(h) + \mathcal{O}\left(\sqrt{\frac{d_{VC}\log(n) + \log(1/\delta)}{n}}\right)$$

Bias-Variance Decomposition

For Mean Squared Error, the error is decomposed as:

$$MSE(\hat{\theta}) = \underbrace{(\mathbb{E}[\hat{\theta}] - \theta)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2]}_{\text{Variance}}$$

Structural Risk Minimization (SRM)

To prevent overfitting, we add a regularization term $\Omega(h)$ to the empirical risk:

$$h^* = \arg\min_{h \in \mathcal{H}} \hat{L}_n(h) + \lambda\Omega(h)$$

Bayes Classifier

- Assume $\mathcal{X}$ is discrete for now
- Suppose $(x, y) \sim \mathcal{D}$
- Want to find $h : \mathcal{X} \to \mathcal{Y}$ that minimizes

$$\mathbb{P}[h(x) \neq y] = \sum_{x \in X} \mathbb{P}[h(X) \neq y | X = x] \cdot \mathbb{P}[X = x]$$

- For every $x \in \mathcal{X}$, set $h(x)$ to minimize $\mathbb{P}[h(X) \neq Y | X = x]$
- $\mathbb{P}[h(X) \neq Y | X = x] = 1 - \mathbb{P}[h(X) = Y | X = x]$
- So, to minimize $\mathbb{P}[h(X) \neq Y | X = x]$, maximize $\mathbb{P}[h(X) = Y | X = x]$
- i.e. choose

$$h(x) = \arg\max_{y \in Y} \mathbb{P}[Y = y | X = x]$$

- **Bayes (Optimal) Classifier** (this is NOT Naïve Bayes!)

Method	Category	Optimization Goal	Key Distinction
MLE	Parameter Estimation	Maximize Likelihood: $P(D\|\Theta)$	Assumes no prior knowledge; purely data-driven.
MAP	Parameter Estimation	Maximize Posterior: $P(\Theta\|D) \propto P(D\|\Theta)P(\Theta)$	Incorporates a prior as a form of regularization to prevent overfitting.
Bayes Optimal	Decision Rule	Minimize True Risk: $R_{\mathcal{D}}(h) = \mathbb{E}[L(h(X), Y)]$	Theoretical "gold standard"; uses full joint distribution without independence assumptions.
Naïve Bayes	Practical Classifier	Maximize: $P(y)\prod_{i=1}^{d} P(x_i\|y)$	Simplifies the problem by assuming conditional independence between features.

Table 1: Comparison of MLE, MAP, Bayes Optimal, and Naïve Bayes.

- **MLE vs. MAP**: While MLE only looks at the training data S , MAP introduces a regularization term (the prior). This is crucial when the number of samples is small relative to the number of parameters.
- **Parameter Estimation vs. Classification**: MLE and MAP are used to find "good" model parameters Θ[cite: 88, 738]. In contrast, a Bayes Classifier is the strategy used to make a prediction y for a new sample x once those parameters are known.
- **Bayes Optimal vs. Naïve Bayes**: The Bayes Optimal classifier is the theoretical limit of performance (Bayes Risk). However, calculating the full joint probability is often impossible. Naïve Bayes makes the "naïve" assumption that features are independent to make the computation feasible.

Bayes Risk

- Denote by h^* the Bayes classifier
- For the Bayes Risk $R_{\mathcal{D}}(h^*)$ it holds $R_{\mathcal{D}}(h^*) \leq R_{\mathcal{D}}(h)$ for any H and $h \in \mathcal{H}$

Batch Learning

- In practical cases we do not know f and we need to estimate h from the data
- **Learner's input:** Training data, $S = \{(x_1, y_1) \dots (x_m, y_m)\} \in (\mathcal{X} \times \mathcal{Y})^m$
- **Learner's output:** A prediction rule, $h : \mathcal{X} \to \mathcal{Y}$
- **Goal:** h should be correct on future examples

IID condition

- **Key assumption:** Data $(x_1, y_1), \dots, (x_n, y_n)$ are identically and independently distributed (i.i.d.) random labeled examples with distribution $\mathcal{D}$
- This assumption is the connection between what we've seen in the past to what we expect to see in the future

Empirical Risk:

Calculate empirical risk by benchmarking prediction against ground truth

$$R_S(h) = \frac{1}{m} \sum_{i=1}^{m} \mathbb{I}[h(x_i) \neq y_i]$$

Can any algorithm always provide a non-trivial guarantee on the error of the predictor it returns? Can we prove, for a fact, that this AI will actually work on data it hasn't seen before? ı**No:**some assumptions are required

Probably Approximately Correct (PAC) Learning

- Learner doesn't know D and the Bayes predictor.
- Learner receives accuracy ϵ and confidence δ
- Learner can ask for training data S containing m(ϵ, δ) examples
- Learner should output h such that:

$$\mathbb{P}[R_{\mathcal{D}}(h) \leq \epsilon] \geq 1 - \delta$$

- PAC = **Probably** (prob $\geq 1 - \delta$) **Approximately** (accuracy ϵ) **Correct**

Definition (PAC learnability)

A hypothesis class $\mathcal{H}$ is PAC learnable if there exists a function $m_{\mathcal{H}} : (0,1)^2 \to \mathbb{N}$ and a learning algorithm with the following property:

- for every $\epsilon, \delta \in (0,1)$
- for every distribution $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$ such that the realizability assumption holds in $\mathcal{H}$

when running the learning algorithm on $m \geq m_{\mathcal{H}}(\epsilon, \delta)$ i.i.d. examples generated by $\mathcal{D}$, the algorithm returns a hypothesis h such that, with probability of at least $1 - \delta$ (over the choice of the examples), $R_{\mathcal{D}}(h) \leq \epsilon$

Figure 4: PAC

Realizability Assumption

- Assume there exists $h^* \in \mathcal{H}$ (class of functions) such that $R_{\mathcal{D}}(h^*) = 0$ (Implies h^* is the Bayes classifier)

Analysis of a Consistent Classifier

- For a **finite** hypothesis class $\mathcal{H}$, with probability $1 - \delta$:

$$R_{\mathcal{D}}(h) \leq \epsilon$$

- Sample complexity requirement:

$$m \geq \frac{\ln(|\mathcal{H}|/\delta)}{\epsilon}$$

(page 25-26-27 lecture 3 Learning Theory proof of this formula)

Shattering & VC-dimension

- **Shattering:** $\mathcal{H}$ shatters S if all $2^{|S|}$ possible classifications are achievable using concepts in $\mathcal{H}$
- **VC-dimension:** The cardinality of the largest set S that can be shattered by $\mathcal{H}$
- If infinite sets can be shattered, $VCdim(\mathcal{H}) = \infty$

Discretization Trick

- In practice, algorithms use floating point (e.g., 32 bits per parameter)
- This makes the hypothesis class actually finite!
- **Rule of thumb:** Take a number of samples at least 10 times the number of parameters

Underfitting and Overfitting

- **Underfitting zone:** High training and generalization error
- **Overfitting zone:** Low training error but high generalization gap
- **Optimal Capacity:** The point where generalization error is minimized

Train - Validation - Test

- **Training set:** Apply algorithm with different parameters
- **Validation set:** Choose the best h^* from $\mathcal{H}$
- **Test set:** Estimate the true error of h^*
- **CRITICAL:** Never use the test set for parameter selection!

Standard Loss Functions

- **Classification (0/1 Loss):** $L_{0/1}(\hat{y}, y) = \begin{cases} 0 & \hat{y} = y \\ 1 & \text{otherwise} \end{cases}$
- **Regression (Squared Loss):** $L(\hat{y}, y) = (\hat{y} - y)^2$

Ordinary Least Squares (OLS)

- Take gradient with respect to w:

$$\nabla_w \|y - Xw\|_2^2 = 2X^\top(Xw - y)$$

- Set to zero: $(X^\top X)w = X^\top y$
- If $X^\top X$ is invertible:

$$w_{ols} := (X^\top X)^{-1} X^\top y$$

Ridge Regression

- Objective: $\frac{1}{2}\|y - Xw\|_2^2 + \frac{\alpha}{2}\|w\|_2^2$
- The regularizer α prevents the model from becoming too complex
- **Solution:**

$$w = (X^\top X + \alpha I)^{-1} X^\top y$$

  [cite: 901]

Learning Theory Summary

- Learning is PAC (Probably Approximately Correct)
- ERM provides generalization guarantees under certain conditions
- Cross-validation is essential for model selection
- Regularization (e.g., Ridge) controls overfitting